THREE WAYS TO CHANGE A MODEL'S BEHAVIOUR

LoRA vs QLoRA vs Full Fine-Tuning

The memory math, the quality gap and the one that actually fits on the card you own. With the numbers behind each claim.

16 bytes per param | rank r | one 24GB card

Al Coach John FOLLOW | PROITBRIDGE Strive For Better Future

REPOST

All Three Change The Same Thing

MENTAL MODEL

Fine tuning edits weights. The three methods differ only in how many weights they edit and at what precision they hold them.

FULL FINE TUNING

EVERY WEIGHT UPDATED, EVERY STEP

LoRA

BASE FROZEN AT BF16 ADAPT

QLoRA

BASE FROZEN AND QUANTIZED TO 4 BIT ADAPT

The green sliver is the only thing that receives gradients in LoRA and QLoRA

100%
trainable, full fine tuning

<1%
trainable, LoRA and QLoRA

4 bit
the frozen base, QLoRA only

Trainable parameter count drives almost everything: memory, training speed, storage per variant and how easily you can serve many versions at once.

Full Fine Tuning

EVERY WEIGHT

The original approach. Continue training the whole model on your data, updating every parameter it has.

STRENGTHS

- Highest capacity to change behaviour
- Best for teaching a new domain or language
- No inference overhead at all
- The reference point everything is measured against

COSTS

- Roughly 16 bytes per parameter of memory
- Multi GPU for anything past a small model
- A full model copy per variant, tens of GB
- Catastrophic forgetting is a real risk

100%	~112 GB	14 GB
of parameters trainable	to train a 7B model	stored per variant

⚠ Notice the second figure. Training a 7B model needs roughly eight times the memory of running it, which is why full fine tuning surprises people.

Where 16 Bytes Per Parameter Goes

THE MATH

Weights are the smallest part of a training run. The optimizer state is what actually fills the card.



bf16 weights, 2 bytes · gradients, 2 bytes · Adam moments, 8 bytes · fp32 master, 4 bytes

```
# mixed precision training with Adam
bytes_per_param = 2 + 2 + 8 + 4 = 16

# 7B model, weights and states only
7e9 * 16 = 112 GB     # plus activations on top

# the same model just running inference
7e9 * 2  = 14 GB      # 8x smaller
```

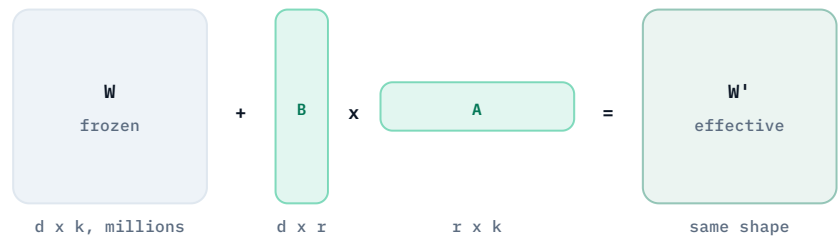
Activation memory sits on top of all of this and scales with batch size and sequence length. Gradient checkpointing trades compute to shrink it.

LoRA, The Low Rank Idea

ADAPTERS

Freeze the model. Learn a small correction beside each big weight matrix, built from two thin matrices multiplied together.

ONLY THE GREEN MATRICES ARE TRAINED. THEY ARE TINY.




```
# the whole method in one line
h = x @ W + (alpha / r) * (x @ A.T @ B.T)
#       ^ frozen        ^ trainable, rank r
```

r = 16
typical rank, tiny beside d and k

0.2%
of parameters actually trained

merge
fold back in, zero overhead

🔑 The insight: the useful change to a big weight matrix is usually low rank. You do not need millions of new numbers to express it, you need a few hundred thousand.

The Knobs That Matter HYPERPARAMETERS

Four settings do almost all the work.

SETTING	WHAT IT CONTROLS	SENSIBLE START
`r`	capacity of the adapter, the rank	16 to 32
`lora_alpha`	how strongly the adapter is applied	2 x r
`target_modules`	which layers get an adapter	all linear layers
`lora_dropout`	regularisation on small datasets	0.05
`learning_rate`	higher than full fine tuning needs	1e-4 to 2e-4

01 **Target every linear layer,** not just attention. Including the MLP projections is consistently better and costs little.

02 **Raise r for new knowledge,** keep it low for style and format. Rank 8 can teach tone, rank 64 for a new domain.

03 **Do not copy a learning rate from full fine tuning.** LoRA wants roughly ten times higher.

> If results are poor, change the data before you change the rank. Rank is rarely the binding constraint, and data quality almost always is.

QLoRA, LoRA On A Compressed Base

4 BIT

Take LoRA and quantize the frozen base model to 4 bit. The adapters stay in higher precision, because they are the part that learns.



INGREDIENT	WHAT IT DOES
NF4	a 4 bit format spaced for normally distributed weights
Double quantization	quantizes the scale factors too, saving more
Paged optimizers	spills optimizer state to CPU on a memory spike

This is the technique that put fine tuning on a single consumer card. A 7B model becomes trainable in well under 24 GB.

Side By Side MATRIX

One table. Every row below is a decision someone has to defend at some point.

DIMENSION	FULL FT	LORA	QLORA
Trainable params	100%	under 1%	under 1%
Memory need	highest	moderate	lowest
Training speed	slowest	fast	slightly slower
Artifact size	tens of GB	tens of MB	tens of MB
Quality ceiling	highest	very close	very close
New knowledge	best	limited	limited
Style and format	great	great	great
Forgetting risk	real	low	low
Serve many variants	hard	easy	easy

Read the fifth and sixth rows together. Adapters give up almost nothing on behaviour, and give up a lot on teaching the model things it has never seen.

What You Can Train On What

FIT TABLE

Approximate, at a modest sequence length and batch size. Enough to decide which method you are even allowed to consider.

MODEL	FULL FT	LORA	QLORA
3B	1 x 80GB	1 x 24GB	1 x 12GB
7B to 8B	2 x 80GB	1 x 48GB	1 x 24GB
13B	4 x 80GB	1 x 80GB	1 x 24GB
32B	8 x 80GB	2 x 80GB	1 x 48GB
70B	16 x 80GB	4 x 80GB	2 x 80GB

16 x
GPUs for a full 70B fine tune

1 x
GPU for a 13B QLoRA run

~40 MB
a typical adapter on disk

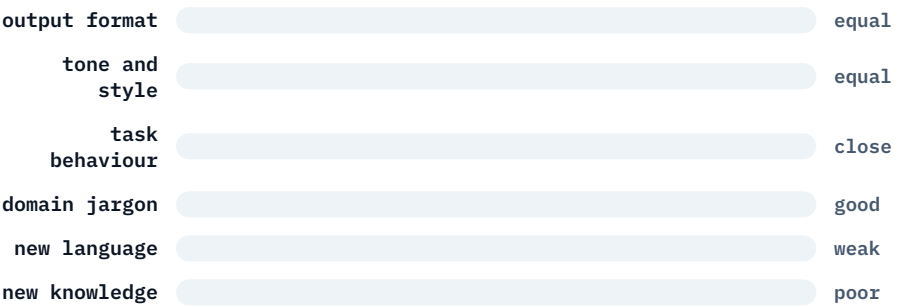
Sequence length changes these numbers a lot. Training at 8k context can double the activation memory compared with 2k, so measure before you book a cluster.

How Much Quality Do You Lose

HONEST

Much less than the memory savings suggest, and the size of the gap depends entirely on what you are trying to teach.

CLOSENESS TO FULL FINE TUNING, BY OBJECTIVE



✓ Teaching behaviour, adapters are enough

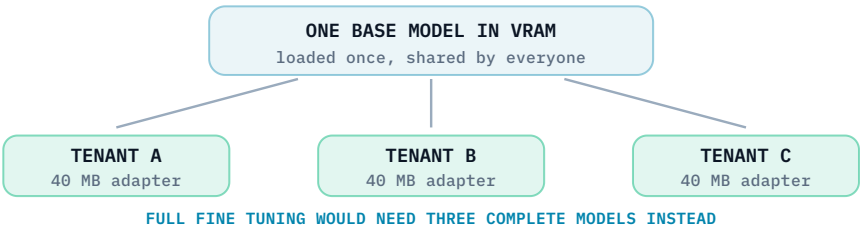
✗ Teaching facts, use retrieval instead

The most common misuse of all three methods: fine tuning to inject facts. Retrieval does that better, cheaper, and stays correct when the facts change.

Why Adapters Win In Production

SERVING

The memory saving during training gets the attention. The serving story is arguably the bigger advantage.



01 **Hot swap per request.** Serving engines can apply a different adapter per request against one base in memory.

02 **Ship a fix in megabytes.** Retraining and redeploying an adapter is a very small artifact to move.

03 **Or merge it in.** Fold the adapter into the base weights before deploying and inference overhead becomes exactly zero.

Serve unmerged when you need many variants at once. Merge when you run a single tuned model and want the last few percent of latency.

The Method Matters Less Than The Data

UNCOMFORTABLE

Teams spend weeks choosing between these three and an afternoon on the dataset. The returns are the other way around.

500	1 to 3	1
good examples beat 50,000 scraped ones	epochs, more usually overfits	bad label pattern can dominate

01 **Consistency beats volume.** The model learns the pattern you repeat, including the mistakes you repeat.

02 **Match production exactly.** Same system prompt, same input shape, same output format as the real thing.

03 **Hold out a real test set.** Split before you start, and never look at it while tuning.

04 **Read a hundred examples by hand.** Every serious practitioner does this, and it always finds something.

If a LoRA run disappoints, the dataset is the first suspect and the rank is close to last. Changing r rarely rescues bad data.

A QLoRA Run, End To End CODE

The configuration that covers most cases. Two objects and a trainer, nothing exotic.

```python
from transformers import BitsAndBytesConfig
from peft import LoraConfig, get_peft_model

quant = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True)

lora = LoraConfig(
    r=32, lora_alpha=64, lora_dropout=0.05,
    target_modules="all-linear",
    task_type="CAUSAL_LM")

model = get_peft_model(base, lora)
model.print_trainable_parameters()   # expect under 1%
```

✓ Always print the trainable percentage

Save adapters, not the merged model

If the trainable percentage prints as zero, your target modules did not match any layer names. This is the single most common silent failure in a LoRA setup.

Which One, And When DECISION

Most teams should reach for QLoRA first and only move up if an eval tells them to.

Q **QLoRA.** One GPU, a big base model, a narrow task. This is the default and it covers most real work.

L **LoRA.** You have memory to spare and want the fastest training with slightly cleaner numerics than 4 bit.

F **Full fine tuning.** A new language or domain, a small base model, a real cluster, and an eval showing adapters are not enough.

P **None of them.** Try a better prompt and retrieval first. A surprising share of fine tuning projects should have been a prompt change.

QLoRA is the right default in 2026

Full FT needs a cluster and a reason

The gap between LoRA and QLoRA is small enough that hardware, not quality, should decide between those two.

The order that saves the most time: **prompt, then retrieval, then QLoRA, then full fine tuning.** Each step costs roughly ten times the one before it.

Seven Ways Runs Go Wrong PITFALLS

Each of these produces a model that trains without error and then disappoints in production.

MISTAKE	CONSEQUENCE
Fine tuning to add facts	hallucinations, stale data
Training format differs from serving	quality collapses in production
Too many epochs	memorised, not generalised
Adapters only on attention layers	leaves quality on the table
Full fine tuning learning rate	adapter barely moves
No held out test set	no idea whether it worked
Never checked the base model first	tuned a problem prompting solved

Train on the exact production template

Test the base model first, every time

⚠ Watch the second row. If training examples use a different template than production, you have taught the model the wrong thing perfectly.

Proving The Tune Worked

EVALUATION

Training loss going down proves the optimizer works. It says nothing about whether the model is better at your job.

01 **Compare against the untuned base.** With the same prompt. A surprising number of tunes lose to good prompting.

02 **Score the task, not the loss.** Accuracy, schema validity, or whatever your product actually cares about.

03 **Check for forgetting.** Run a small general capability set to confirm you did not break unrelated behaviour.

04 **Test the merged artifact.** Merge the adapter, then evaluate that, because merging can shift numerics slightly.

Beat the base by a clear margin

Or throw the run away, honestly

Build the eval set before the training script. It takes an afternoon and it is the only thing that turns fine tuning from a hobby into engineering.

One Line Each

RECAP TABLE

The sentences you would use to explain each method to a colleague in a corridor.

TOPIC	THE LINE THAT MATTERS
Full FT	every weight moves, roughly 16 bytes per parameter
LoRA	freeze the base, learn a thin low rank correction
QLoRA	the same, on a base quantized to 4 bit NF4
Memory	optimizer state, not weights, fills the card
r and alpha	start at 16 to 32, alpha at twice r
Targets	all linear layers beats attention only
Quality	adapters match full FT on behaviour, not on facts
Serving	one base, many adapters, swapped per request
Data	500 consistent examples beat 50,000 scraped
Order	prompt, retrieval, QLoRA, then full fine tuning

Keep one row, keep the last. **Most disappointment comes from skipping the two cheap steps** before it.

Five Questions Before You Train DECIDE

Answer these honestly and you will either start the right run or save yourself the whole project.

Q1 **Can a better prompt do it?** Test properly with few shot examples first. This settles more cases than anyone admits.

Q2 **Is it a knowledge problem?** Then build retrieval. Fine tuning is the wrong tool for facts that change.

Q3 **Do you have 500 clean examples?** If not, that is the project. Collect and clean before you configure anything.

Q4 **How much GPU do you actually have?** One card means QLoRA. That is not a compromise, it is the normal answer.

Q5 **Did the adapter close the gap?** If yes, ship it. Only escalate to full fine tuning when an eval demands it.

Question one and two eliminate most fine tuning projects, and that is a good outcome. The cheapest run is the one you correctly decided not to start.

The Terms You Will Meet

REFERENCE

Everything on a typical fine tuning config screen, in plain words.

TERM	PLAIN MEANING
adapter	the small trained file you load beside a base model
rank (r)	how much capacity the adapter has to change things
alpha	a scaling factor on how strongly it is applied
merge	folding the adapter into the base weights permanently
NF4	4 bit format spaced for normal distributions
double quant	quantizing the quantization scales as well
paged optimizer	spilling optimizer state to CPU during spikes
grad checkpointing	recompute activations to save memory
catastrophic forgetting	losing old abilities while learning new ones

Two of these are pure memory tricks with no effect on quality. Gradient checkpointing and paged optimizers are free wins whenever you are close to the limit.